INTERNSHIP · TODOWEBAPI

Working with the database

How our data is stored, how we talk to SQLite with plain SQL, and how the database code is wired into the rest of the project.

SQLite Microsoft.Data.Sqlite Plain SQL, no Entity Framework

AGENDA

What we will cover

PART 1 · THE DATABASE

1 **Why a database?**
The JSON file and what was wrong with it

2 **Structure**
Two tables and the one-to-many link

3 **One query, step by step**
Connection, command, parameters, execute

4 **Safety: parameters**
How we stop SQL injection

PART 2 · HOW IT IS WIRED

5 **Layers**
Program → Services → Repositories → Database

6 **Program.cs**
Startup and the routes

7 **One request, end to end**
Following `PUT /todos/1/done` through the files

8 **Try it yourself**
Look inside todos.db and small exercises

From a JSON file to SQLite

BEFORE: TASKS.JSON

- All tasks live in one list in memory.
- At startup we read the whole file.
- On every change we rewrite the whole file.
- We choose the Id ourselves (`GetNextId`).
- No users. Anyone can change anything.

NOW: TODOS.DB

- The data lives in a database file.
- Each query reads or writes only what it needs.
- The database chooses the Id (`AUTOINCREMENT`).
- The database checks rules: `NOT NULL`, `UNIQUE`, foreign key.
- Users own their todos and prove it with a password.

The old JSON code is still in `src/Repositories/JsonTaskRepository.cs`, so you can compare the two versions.

Two tables, one link

Users	
PK **Id**	INTEGER AUTOINCREMENT
Name	TEXT NOT NULL UNIQUE
Password	TEXT NOT NULL

1 — many

Todos.UserId = Users.Id

Todos	
PK **Id**	INTEGER AUTOINCREMENT
Title	TEXT NOT NULL
IsDone	INTEGER NOT NULL DEFAULT 0
DueDate	TEXT NULL
FK **UserId**	INTEGER NOT NULL

One user has many todos. Each todo has exactly one user. The link is the `UserId` column: it holds the `Id` of a row in `Users`.

PK (primary key): the unique number of each row. **FK** (foreign key): `REFERENCES Users(Id)`. The database refuses a todo whose user does not exist.

Creating the tables at startup

```
// src/Repositories/Database.cs
public static void InitializeDatabase()
{
    using (SqliteConnection connection = OpenConnection())
    {
        SqliteCommand command = connection.CreateCommand();
        command.CommandText = @"
            CREATE TABLE IF NOT EXISTS Users (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Name TEXT NOT NULL UNIQUE,
                Password TEXT NOT NULL
            );

            CREATE TABLE IF NOT EXISTS Todos (
                Id INTEGER PRIMARY KEY AUTOINCREMENT,
                Title TEXT NOT NULL,
                IsDone INTEGER NOT NULL DEFAULT 0,
                DueDate TEXT NULL,
                UserId INTEGER NOT NULL REFERENCES Users(Id)
            );";
        command.ExecuteNonQuery();
    }
}
```

- `Main` calls it **once**, when the app starts.
- `IF NOT EXISTS`: if the table is already there, nothing happens. Your data is safe.
- No `todos.db` file yet? SQLite creates it.
- It creates **empty** tables. All data comes in through the API.

Try it: stop the app, delete `todos.db`, start again. The tables come back, empty.

C# types and SQLite types

TODOITEM PROPERTY	C# TYPE	COLUMN	HOW WE STORE IT
`Id`	`int`	`INTEGER`	The database chooses it.
`Title`	`string`	`TEXT`	As it is.
`IsDone`	`bool`	`INTEGER`	SQLite has no true/false. We save **1** or **0**. `task.IsDone ? 1 : 0` and back with `reader.GetInt32(2) == 1`
`DueDate`	`DateTime?`	`TEXT`	ISO 8601 text, e.g. `2026-12-09T10:29:20.1250000Z`, or **NULL** for no date. `DueDateToDbValue` and back with `DateTime.Parse`
`UserId`	`int`	`INTEGER`	Foreign key to `Users(Id)`.

Watch out: SQLite does **not** check foreign keys unless you ask. So `Database.OpenConnection()` runs `PRAGMA foreign_keys = ON;` on **every** new connection.

Anatomy of a query: `TaskRepository.GetById`

```
public static TodoItem? GetById(int id)
{
    using (SqliteConnection connection = Database.OpenConnection()) 1
    {
        SqliteCommand command = connection.CreateCommand(); 2
        command.CommandText = "SELECT Id, Title, IsDone, DueDate, UserId FROM Todos WHERE Id = $id;";
        command.Parameters.AddWithValue("$id", id); 3

        using (SqliteDataReader reader = command.ExecuteReader()) 4
        {
            if (reader.Read())
            {
                return ReadTaskFromRow(reader); 5
            }
        }
    }

    return null;   // no row with this id
}
```

1. **Open** a connection. `using` closes it at the end, even after an error.
2. **Write** the SQL in a command.
3. **Fill** the parameter `$id` with the value.
4. **Run** it. A reader walks over the rows.
5. **Turn** the row into a `TodoItem`.

Three ways to run a command

METHOD	WHAT COMES BACK	WHERE WE USE IT
`ExecuteReader()`	Rows. Read them one by one with `reader.Read()`.	`GetAll`, `GetById`, `GetByUserId`, `GetByName`
`ExecuteScalar()`	One single value: first column of the first row.	`Add`: returns the new Id
`ExecuteNonQuery()`	Nothing useful. It just does the work.	`Update`, `Delete`, `CREATE TABLE`, `PRAGMA`

```
// TaskRepository.Add
command.CommandText = @"
  INSERT INTO Todos
    (Title, IsDone, DueDate, UserId)
  VALUES
    ($title, $isDone, $dueDate, $userId);
  SELECT last_insert_rowid();";
...
return Convert.ToInt32(
    command.ExecuteScalar());
```

The database picks the new Id. `last_insert_rowid()` asks for it, and `ExecuteScalar` brings it back to C#.

Never glue user input into SQL

NEVER DO THIS (NOT IN OUR CODE)

```
string sql =
  "SELECT * FROM Users WHERE Name = '"
  + name + "';";
```

A caller sends this name:

```
x' OR '1'='1
```

The SQL becomes:

```
SELECT * FROM Users WHERE Name = 'x' OR '1'='1';
```

→ returns **every** user. The input became SQL code.

WHAT WE DO: PARAMETERS

```
// UserRepository.GetByName
command.CommandText =
  "SELECT Id, Name, Password FROM Users
   WHERE Name = $name;";
command.Parameters.AddWithValue(
  "$name", name);
```

- The SQL text never changes.
- The database treats `$name` as **data**, never as SQL code.
- `x' OR '1'='1` is just a strange name. No user has it.

Rule: every value goes in with a $parameter. No string concatenation, ever.

Turning a row into an object

```
// TaskRepository.cs
static TodoItem ReadTaskFromRow(SqliteDataReader reader)
{
    TodoItem task = new TodoItem(reader.GetString(1));
    task.Id = reader.GetInt32(0);

    task.IsDone = reader.GetInt32(2) == 1;

    if (!reader.IsDBNull(3))
    {
        task.DueDate = DateTime.Parse(
            reader.GetString(3), ...);
    }

    task.UserId = reader.GetInt32(4);
    return task;
}
```

INDEX	COLUMN IN THE SELECT
0	`Id`
1	`Title`
2	`IsDone`
3	`DueDate`
4	`UserId`

- Written **once**, used by three queries.
- The numbers must match the order in the `SELECT`.
- `IsDBNull` checks for NULL first: a task without a due date.

All database operations

`TaskRepository` · Todos table

METHOD	SQL
`GetAll()`	`SELECT … ORDER BY Id`
`GetById(id)`	`SELECT … WHERE Id = $id`
`GetByUserId(userId)`	`SELECT … WHERE UserId = $userId`
`Add(task)`	`INSERT … ; SELECT last_insert_rowid()`
`Update(task)`	`UPDATE … WHERE Id = $id`
`Delete(id)`	`DELETE … WHERE Id = $id`

`UserRepository` · Users table

METHOD	SQL
`GetAll()`	`SELECT … ORDER BY Id`
`GetById(id)`	`SELECT … WHERE Id = $id`
`GetByName(name)`	`SELECT … WHERE Name = $name`
`Add(user)`	`INSERT … ; SELECT last_insert_rowid()`

Every method opens its own connection in a `using` block, runs **one** query, and closes the connection.

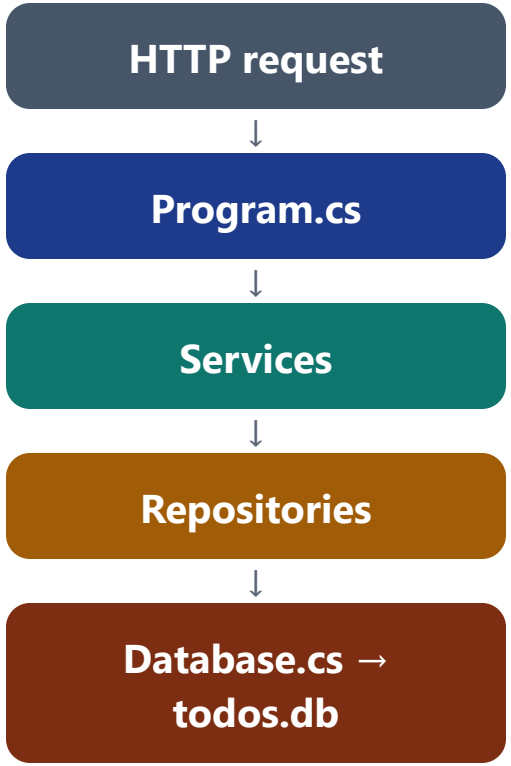
Part 2

How it is wired into the project

The SQL lives in the repositories. Now: who calls them, in which order, and why each layer has only one job.

5 · LAYERS

Each layer has one job



Swagger UI, curl, or a browser sends e.g. `PUT /todos/1/done`.

Wiring. Creates the tables at startup and connects each URL to one method.
`src/Program.cs`

Rules. Checks the input and the password, and chooses the answer: 200, 400, 401, 404, 409.
`src/Services/TaskService.cs` · `UserService.cs`

SQL only. No rules, no status codes. Just read and write rows.
`src/Repositories/TaskRepository.cs` · `UserRepository.cs`

Connection. Opens connections, turns on foreign keys, creates the tables.
`src/Repositories/Database.cs`

Models (`TodoItem`, `User`, DTOs in `src/Models`) are the data that travels between all layers.

Program.cs connects everything

```
public static void Main(string[] args)
{
    ... // builder, Swagger

    // Before (JSON): tasks = LoadTasks();
    Database.InitializeDatabase();

    // ---------- Task endpoints ----------
    app.MapGet("/todos",           TaskService.GetAllTasks);
    app.MapGet("/todos/open",      TaskService.GetOpenTasks);
    app.MapGet("/todos/search",    TaskService.SearchTasks);
    app.MapGet("/todos/{id}",      TaskService.GetTaskById);
    app.MapPost("/todos",          TaskService.AddTask);
    app.MapPut("/todos/{id}",      TaskService.UpdateTask);
    app.MapPut("/todos/{id}/done", TaskService.MarkTaskDone);
    app.MapDelete("/todos/{id}",   TaskService.DeleteTask);

    // ---------- User endpoints ----------
    app.MapPost("/users",           UserService.AddUser);
    app.MapGet("/users",            UserService.GetAllUsers);
    app.MapGet("/users/{id}",       UserService.GetUserById);
    app.MapGet("/users/{id}/todos", UserService.GetTasksOfUser);

    app.Run();
}
```

STEP 1 · STARTUP

`InitializeDatabase()` makes sure the tables exist **before** the first request arrives.

STEP 2 · ROUTES

Each line says: **this method + this URL → this C# method**. .NET fills the parameters (`id` , the JSON body) for us.

Program.cs never touches SQL. If you want to know **what** a URL does, start here and follow the method name.

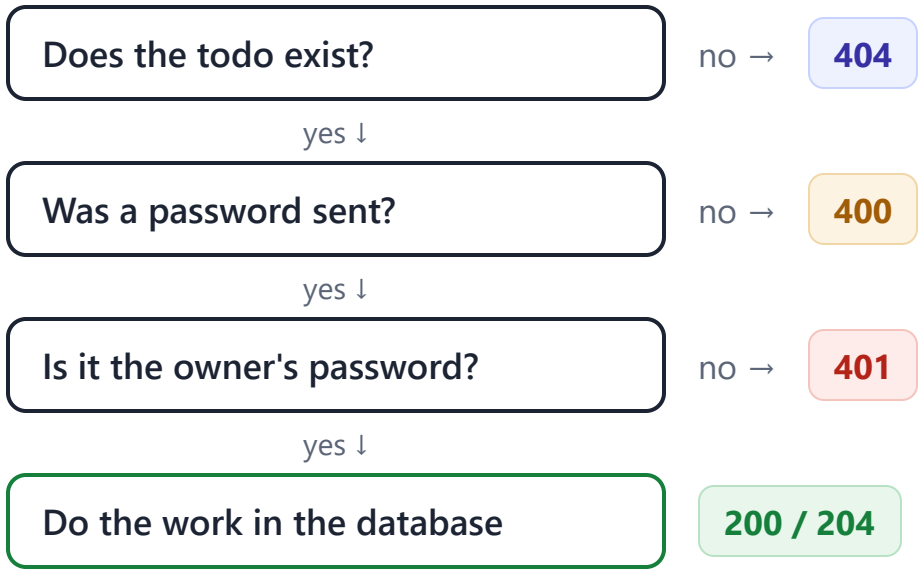
7 · ONE REQUEST, END TO END

PUT /todos/1/done body: { "password": "ana123" }

1	Program.cs	The route `/todos/{id}/done` calls `TaskService.MarkTaskDone(1, body)`.
2	TaskService	Asks `TaskRepository.GetById(1)`. It runs `SELECT … WHERE Id = $id`. No row? **404 Not Found**
3	TaskService	Is `body.Password` empty? **400 Bad Request**
4	UserService	`IsCorrectPassword(task.UserId, "ana123")` reads the owner with `UserRepository.GetById` and compares. Wrong? **401 Unauthorized**
5	Models/TodoItem	`task.MarkDone()` sets `IsDone = true`, only in memory for now.
6	TaskRepository	`Update(task)` runs `UPDATE Todos SET … IsDone = $isDone … WHERE Id = $id`. Now it is saved.
7	TaskService	Returns the task as JSON. **200 OK**

7 · ONE REQUEST, END TO END

Same checks, same order, every time



Used by `PUT /todos/{id}`, `PUT /todos/{id}/done` and `DELETE /todos/{id}`.

`POST /todos` is similar: title and password are required (**400**), the user must exist (**404**), the password must match (**401**), then **201 Created**.

> The password check exists in **one place only**: `UserService.IsCorrectPassword`. All four endpoints call it.

Reading data (`GET`) needs no password.

8 · TRY IT YOURSELF

Look inside todos.db

In VS Code, with the **SQLite** extension: right-click `todos.db` → **Open Database**. Then **New Query** and run:

```
-- the todos of user 1
SELECT * FROM Todos WHERE UserId = 1;

-- the one-to-many link in action
SELECT Todos.Title, Todos.IsDone, Users.Name
FROM Todos
JOIN Users ON Todos.UserId = Users.Id;
```

The grid does not refresh by itself. Run the query again after you call the API.

Exercises

- Find the line in `UserRepository.GetByName` that protects against SQL injection.
- Mark a todo as done in Swagger. Which column changed, and to what value?
- Why does `Users` show the passwords? What would a real app store instead?
- Plan a new endpoint `GET /todos/done`. Which three files would you change, and what goes in each?

SUMMARY

What to remember

PROGRAM.CS

Wires URLs to methods and creates the tables at startup.

SERVICES

Hold the rules and choose the status code. No SQL.

REPOSITORIES

Hold the SQL. One small method per query. No rules.

- Two tables, linked by `Todos.UserId → Users.Id`. One user, many todos.
- Every query: open connection → command → **parameters** → execute → close.
- The database chooses the Id. We read it back with `last_insert_rowid()`.

Coming next: dependency injection and interfaces. One line in Program.cs will switch the app between the JSON file and SQLite. Later: storing a hash instead of a plain password.